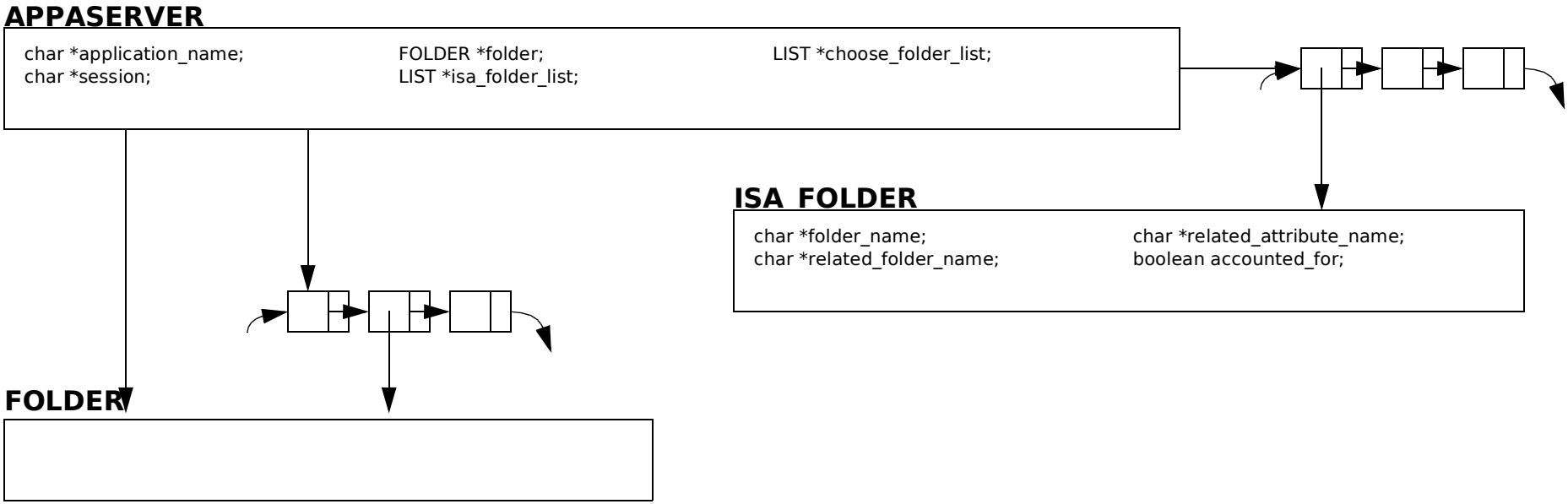
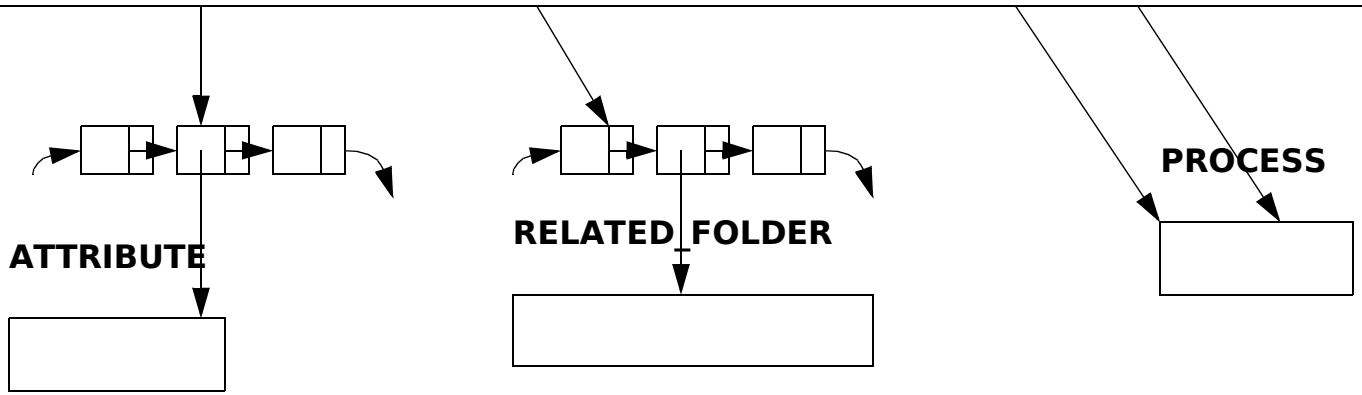
APPASERVER Data Structure

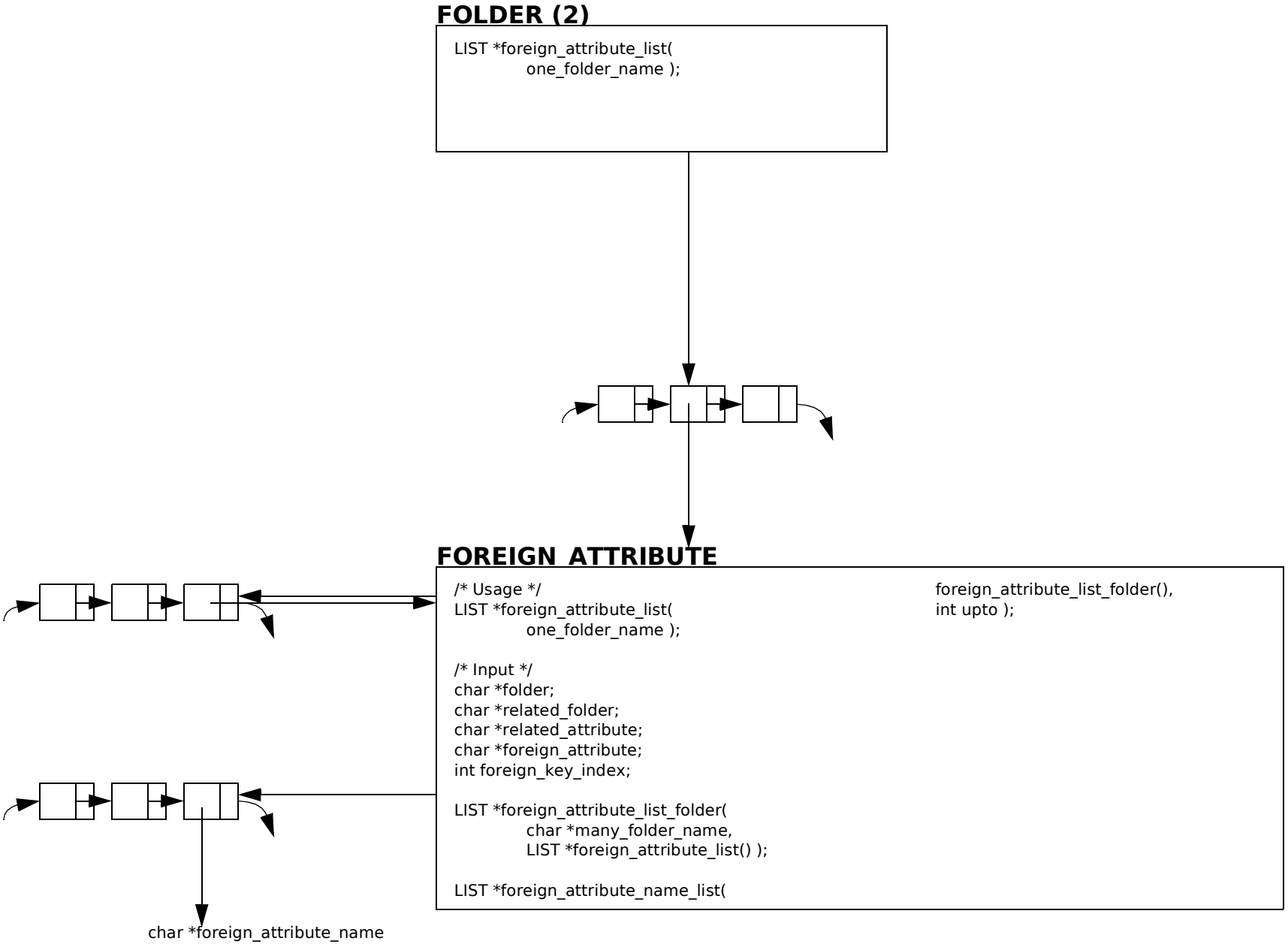
July 19, 2021



FOLDER (1)

```
/* Usage */
FOLDER *folder_fetch(
        char *folder_name,
        boolean fetch_attribute_list,
        boolean fetch_one2m_relation_list,
        boolean fetch_mto1_relation_list,
        boolean fetch_mto1_isa_relation_list );

/* Input */
char *application_name;
char *folder_name;
int insert_rows_number;
long row_count;
boolean insert_permission;
boolean update_permission;
boolean lookup_permission;
char *post_change_javascript;
boolean lookup_email_output;
char *session;
char *folder_form;
long row_access_count;
boolean lookup_before_drop_down;
char *notepad;
PROCESS *populate_drop_down_process;
PROCESS *post_change_process;
char *html_help_file_anchor;
LIST *non_primary_attribute_list;
boolean lookup_transmit_output;
boolean row_level_non_owner_forbid;
boolean row_level_non_owner_view_only;
boolean no_initial_capital;
char *schema_name;

/* Process */
LIST *attribute_list(
        folder_name );

LIST *folder_primary_attribute_name_list(
        attribute_list() );

LIST *primary_attribute_name_list(
        attribute_list() );

DICTIONARY *primary_data_dictionary;

LIST *folder_mto1_isa_related_folder_list(
        existing_related_folder_list,
        application_name,
        folder_name,
        role_name,
        recursive_level );

LIST *folder_append_isa_attribute_list(
        application_name,
        folder_name,
        folder_mto1_isa_related_folder_list,
        role_name );

LIST *folder_attribute_name_list(
        attribute_list );

LIST *folder_append_isa_attribute_name_list(
        folder_append_isa_attribute_list );
```




FOLDER (2)
LIST *foreign_attribute_list(
one_folder_name);
FOREIGN_ATTRIBUTE
/* Usage */
LIST *foreign_attribute_list(
one_folder_name);
/* Input */
char *folder;
char *related_folder;
char *related_attribute;
char *foreign_attribute;
int foreign_key_index;
LIST *foreign_attribute_list_folder(
char *many_folder_name,
LIST *foreign_attribute_list());
LIST *foreign_attribute_name_list(
foreign_attribute_list_folder(),
int upto);
char *foreign_attribute_name

FOLDER (3)

```
LIST *pair_1tom_related_folder_list;
LIST *mto1_append_isa_related_folder_list;
LIST *mto1_lookup_before_drop_down_related_folder_list;
```

```
FOLDER *folder_load_new() sets:

LIST *mto1_related_folder_list;

void folder_append_mto1_related_folder_list(
        mto1_related_folder_list,
        application_name );

LIST *mto1_recursive_related_folder_list;

void folder_append_one2m_related_folder_list(
        mto1_recursive_related_folder_list,
        application_name );

LIST *mto1_isa_related_folder_list;

void folder_append_mto1_related_folder_list(
        mto1_isa_related_folder_list,
        application_name );

LIST *append_isa_attribute_list;
LIST *mto1_append_isa_related_folder_list;
LIST *attribute_list;
LIST *attribute_name_list;
LIST *append_isa_attribute_name_list;
LIST *primary_attribute_name_list;
LIST *lookup_allowed_attribute_name_list;
LIST *pair_one2m_related_folder_list;
LIST *one2m_related_folder_list;
LIST *one2m_recursive_related_folder_list;
LIST *join_1tom_related_folder_list;

LIST *folder_lookup_attribute_exclude_name_list(
        LIST *append_isa_attribute_list );
```

ATTRIBUTE

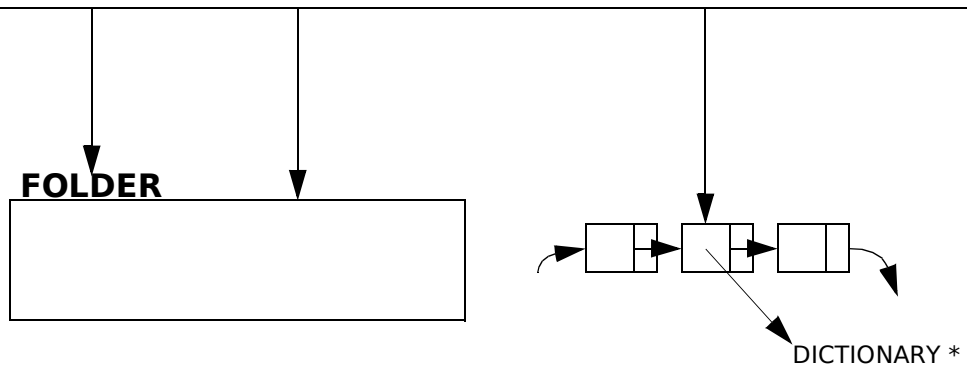
```
/* Input */
char *attribute_name;
char *datatype_name;
DATATYPE *datatype;
int width;
int float_decimal_places;
char *hint_message;
boolean lookup_required;
boolean insert_required;
char *post_change_javascript
char *on_focus_javascript_function
boolean additional_unique_index
boolean additional_index
boolean lookup_histogram_output
boolean lookup_time_chart_output
boolean appaserver;

/* Process */
char *folder_name;
int display_order;
boolean omit_insert_prompt;
boolean omit_insert;
int primary_key_index;
LIST *exclude_permission_list;
```

```
LIST *attribute_fetch_list(
        char *folder_name );

LIST *attribute_primary_name_list(
        attribute_fetch_list(
                char *folder_name ) );
```

DATATYPE

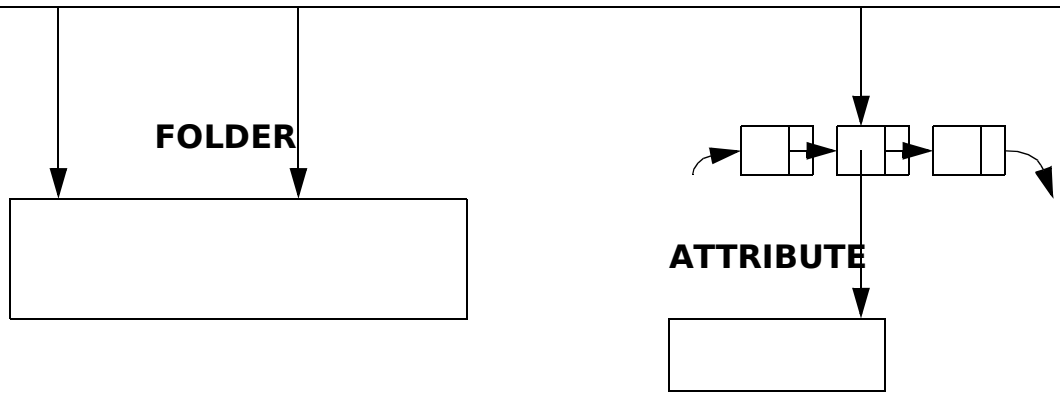
RELATED_FOLDER

```
/* Input */
FOLDER *folder;
FOLDER *one2m_related_folder;
char *related_attribute_name;
boolean relation_type_isa;
int pair_1tom_order;
boolean null_1tom_upon_delete;
boolean ignore_output;
char *hint_message;
boolean copy_common_attributes;
boolean prompt_mto1_recursive;
boolean omit_1tom_detail;
char *join_where_clause;
boolean drop_down_multi_select;
boolean automatic_preselection;

/* Process */
LIST *foreign_attribute_name_list;
LIST *folder_foreign_attribute_name_list;
LIST *parent_foreign_attribute_name_list;
LIST *primary_data_list;
LIST *common_non_primary_attribute_name_list;
LIST *automatic_preselection_dictionary_list;
```



RELATION

```
/* Input */
FOLDER *mto1_folder;
FOLDER *one2m_folder;
char *related_attribute_name;
int pair_1tom_order;
boolean omit_1tom_detail;
boolean prompt_mto1_recursive;
boolean relation_type_isa;
boolean copy_common_attributes;
boolean automatic_preselection;
boolean drop_down_multi_select;
boolean join_1tom_each_row;
boolean omit_lookup_before_drop_down;
boolean ajax_fill_drop_down;
char *hint_message;

/* Process */
LIST *relation_one2m_relation_list(
        char *one2m_folder_name );

LIST *relation_one2m_recursive_relation_list(
        LIST *relation_list /* in only */,
        char *one2m_folder_name );

boolean relation_is_primary_key_subset(
        LIST *foreign_attribute_name_list,
        LIST *mto1_primary_attribute_name_list );

LIST *relation_foreign_attribute_name_list(
        attribute_primary_name_list(
                one2m_folder->
                attribute_list )
                /* primary_foreign_attribute_name_list */,
        related_attribute_name,
        foreign_attribute_name_list(
                foreign_attribute_list_folder(
                        mto1_folder->folder_name
                        foreign_attribute_list() ) );

boolean ignore_output;

char *relation_join_where_clause();
```



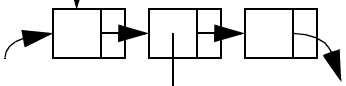
ROW_SECURITY (1)

```
/* Usage */
ROW_SECURITY *row_security_simple_new(
        );

ROW_SECURITY *row_security_detail_new(
        char *application_name,
        enum row_security_state row_security_state,
        LIST *where_attribute_data_list,
        FOLDER *folder,
        LIST *ignore_attribute_name_list
        );

/* Input */
char *state;
FOLDER *folder;
ROLE *role;
char *login_name;
ENTITY *entity;
DICTIONARY *preprompt_dictionary;
DICTIONARY *query_dictionary;
DICTIONARY *sort_dictionary;
LIST *role_update_list;
LIST *forbid_list;
char *attribute_not_null_string;
FOLDER *foreign_login_name_folder;
FOLDER *attribute_not_null_folder;
LIST *ignore_attribute_name_list;

/* Process */
```

```
enum                 {  security_supervisor   security_user
row_security_state   {  regular_supervisor     regular_user
```

FOLDER

```
LIST *lookup_attribute_exclude_name_list;
LIST *update_attribute_exclude_name_list;
```

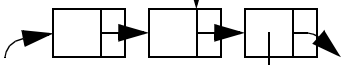


ROW_SECURITY_ROW_FORBID

```
char *forbid_login_name;
char *forbid_full_name;
char *forbid_street_address;
```

ROW_SECURITY (2)

```
/* Process */
LIST *row_security_role_update_list(
        void );
```

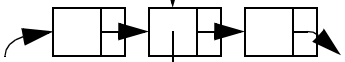


ROW_SECURITY_ROLE_UPDATE

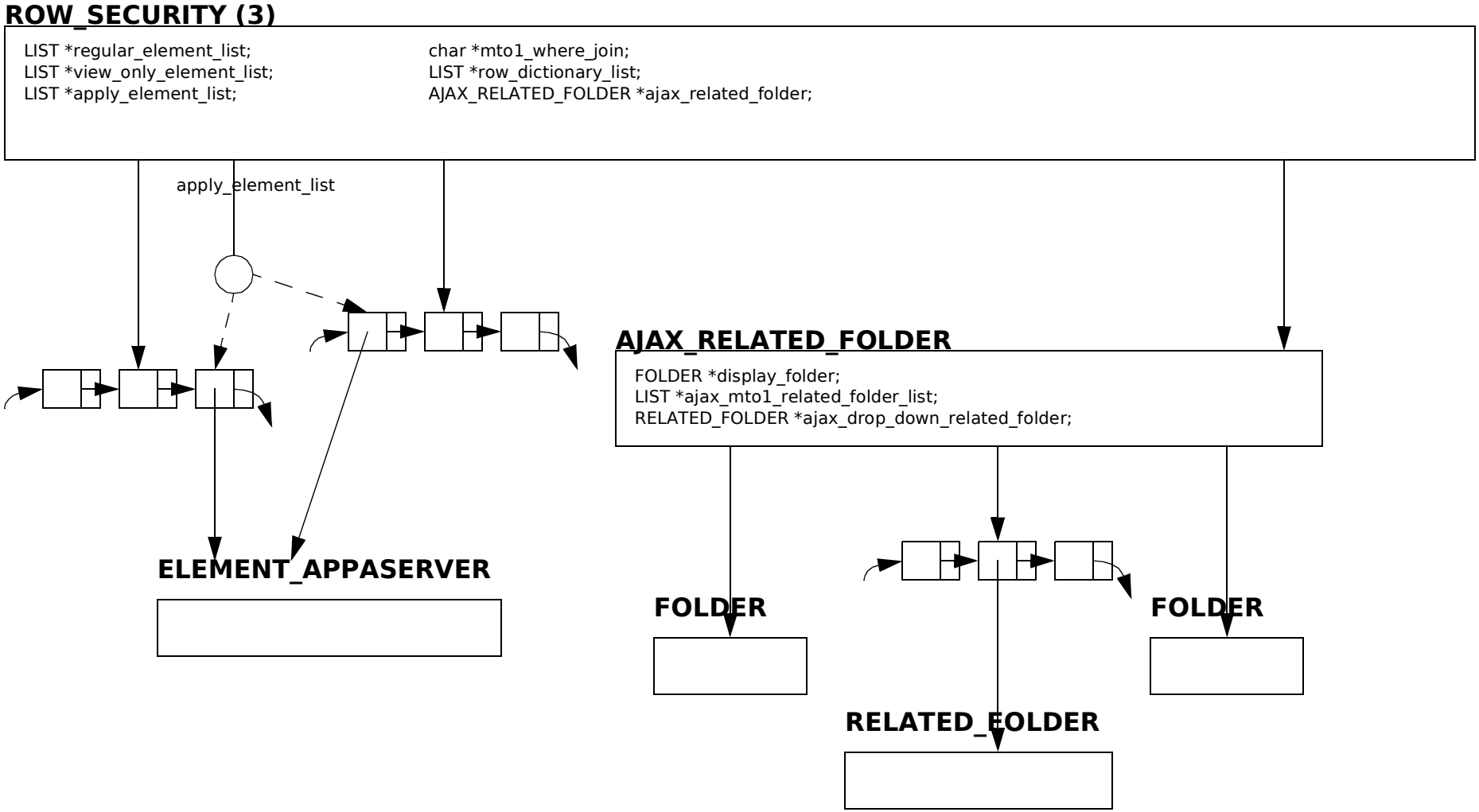
```
/* Usage */
ROW_SECURITY_ROLE_UPDATE *
        row_security_role_update_new(
                char *folder_name,
                char *attribute_not_null_string );

FOLDER *role_update_folder;
char *attribute_not_null_string;

/* Process */
boolean row_security_role_update_attribute_seek(
        attribute_name,
        row_security_role_update_list() );

boolean row_security_role_update_row_view_only(
        char *folder_name,
        FOLDER *role_update_folder,
        enum row_security_state,
        boolean attribute_not_null );
```

FOLDER

```
char *folder_name;
LIST *one2m_recursive_related_folder_list;
```



RELATED_FOLDER


ROW_SECURITY (3)
LIST *regular_element_list;
LIST *view_only_element_list;
LIST *apply_element_list;
char *mto1_where_join;
LIST *row_dictionary_list;
AJAX_RELATED_FOLDER *ajax_related_folder;
apply_element_list
AJAX_RELATED_FOLDER
FOLDER *display_folder;
LIST *ajax_mto1_related_folder_list;
RELATED_FOLDER *ajax_drop_down_related_folder;
ELEMENT_APPASERVER
FOLDER
FOLDER
RELATED_FOLDER

ELEMENT_APPASERVER

```
char *name;                          boolean remember_keystrokes_ok;
enum element_type element_type;      int tab_index;
```

drop_down
push_button
radio_button
notepad
password
non_edit_text
linebreak
text_item
upload_filename
prompt_data
prompt_data_plus_hidden
reference_number
hidden
empty_column
blank
element_date
element_current_date

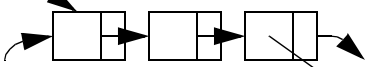
element_time
element_current_time
http_filename
timestamp
table_opening
table_closing
prompt
anchor

```
ELEMENT_DROP_DOWN *drop_down;
ELEMENT_PUSH_BUTTON *push_button;
ELEMENT_RADIO_BUTTON *radio_button;
ELEMENT_NOTEPAD *notepad;
ELEMENT_LINEBREAK *linebreak;
ELEMENT_TEXT_ITEM *text_item;
ELEMENT_PASSWORD *password;
ELEMENT_PROMPT_DATA *prompt_data;
ELEMENT_HIDDEN *hidden;
ELEMENT_UPLOAD_FILENAME *upload_filename;
ELEMENT_HTTP_FILENAME *http_filename;
ELEMENT_ANCHOR *anchor;
ELEMENT_EMPTY_COLUMN *empty_column;
ELEMENT_REFERENCE_NUMBER *reference_number;
ELEMENT_NON_EDIT_TEXT *non_edit_text;
```

APPASERVER Data Structure

LOOKUP_BEFORE_DROP_DOWN

```
FOLDER *base_folder;
LIST *lookup_before_drop_down_folder_list;
boolean first_prelookup;
char *insert_pair_base_folder_name;
char *insert_folder_name;
boolean omit_lookup_before_drop_down;

enum lookup_before_drop_down_state lookup_before_drop_down_state;
```



lookup_not_participating
lookup_participating_not_root
lookup_skipped
lookup_participating_is_root_all_complete

FOLDER

```
char *folder_name;
boolean lookup_before_drop_down;
LIST *mto1_lookup_before_drop_down_related_folder_list;
LIST *pair_1tom_related_folder_list;
```

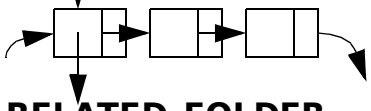
LOOKUP_BEFORE_DROP_DOWN_FOLDER

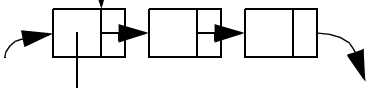
```
char *folder_name;
boolean fulfilled;
```

RELATED_FOLDER

PROMPT_RECURSIVE

```
/* Usage */
PROMPT_RECURSIVE *prompt_recursive_new(
        char *folder_name,
        LIST *mto1_related_folder_list );

/* Input */
char *folder_name;
LIST *mto1_related_folder_list;

/* Process */
LIST *prompt_recursive_folder_list(
        mto1_related_folder_list );

char *prompt_recursive_display(
        PROMPT_RECURSIVE *prompt_recursive );
```





RELATED_FOLDER

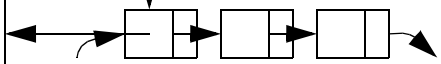
PROMPT_RECURSIVE_FOLDER

```
/* Usage */
LIST *prompt_recursive_folder_list(
        LIST *mto1_related_folder_list );

PROMPT_RECURSIVE_FOLDER *prompt_recursive_folder_new(
        FOLDER *related_folder->folder,
        boolean related_folder->drop_down_multi_select );

/* Input */
FOLDER *folder;
boolean drop_down_multi_select;

/* Process */
LIST *prompt_recursive_mto1_folder_list(
        related_folder->folder->folder_name,
        related_folder->folder->primary_attribute_name_list );
```

FOLDER

PROMPT_RECURSIVE_MTO1_FOLDER

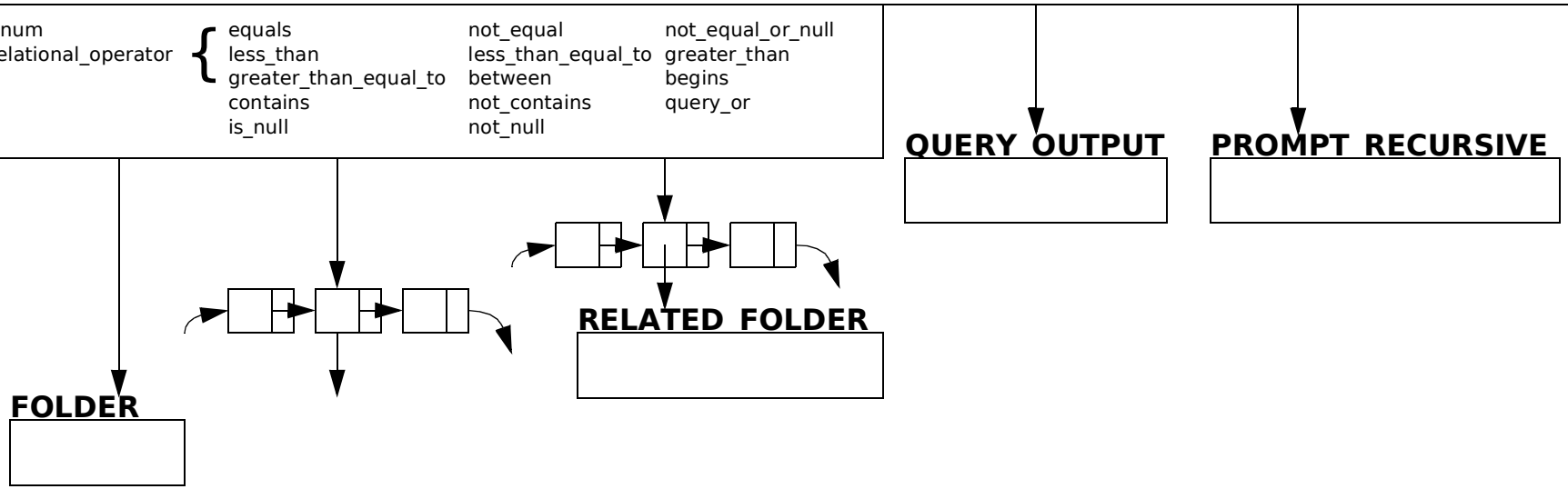
```
/* Usage */
LIST *prompt_recursive_mto1_folder_list(
        char *folder_name,
        LIST *primary_attribute_name_list );

PROMPT_RECURSIVE_MTO1_FOLDER *
        prompt_recursive_mto1_folder_new(
        FOLDER *folder,
        int recursive_level );

FOLDER *folder;
int recursive_level;
```



QUERY

```
/* Usage */
QUERY *query_simple_new(
        DICTIONARY *query_dictionary,
        FOLDER *mto1_folder,
        LIST *ignore_attribute_name_list,
        char *forbid_login_name,
        char *forbid_full_name,
        char *forbid_street_address );

QUERY *query_edit_table_new(
        DICTIONARY *query_dictionary,
        FOLDER *mto1_folder,
        LIST *ignore_attribute_name_list,
        char *forbid_login_name,
        char *forbid_full_name,
        char *forbid_street_address,
        char *attribute_not_null_join,
        char *attribute_not_null_folder_name,
        DICTIONARY *sort_dictionary );

QUERY *query_detail_new(
        LIST *where_attribute_data_list,
        FOLDER *mto1_folder,
        LIST *ignore_attribute_name_list,
        char *forbid_login_name,
        char *forbid_full_name,
        char *forbid_street_address,
        char *attribute_not_null_join,
        char *attribute_not_null_folder_name );

QUERY *query_process_new();

/* Input */
DICTIONARY *query_dictionary;
LIST *where_attribute_data_list;
FOLDER *mto1_folder;
LIST *ignore_attribute_name_list;
char *forbid_login_name;
char *forbid_full_name;
char *forbid_street_address;
DICTIONARY *sort_dictionary;

/* Process */
LIST *mto1_join_folder_list;
LIST *one2m_subquery_related_folder_list;
QUERY *query_output;
PROMPT_RECURSIVE *prompt_recursive;
int max_rows;

QUERY *query_calloc(
        void );

LIST *query_where_attribute_name_list(
        LIST *foreign_attribute_name_list );

char *query_dictionary_operator_name(
        char *attribute_name,
        DICTIONARY *query_dictionary,
        int dictionary_offset,
        char *starting_label );

enum relational_operator query_relational_operator(
        char *query_dictionary_operator_name(),
        char *attribute_datatype );

char *query_sort_prefix_direction_attribute_index(
        DICTIONARY *sort_dictionary );

char *query_attribute_name_list_order(
        char *folder_name,
        LIST *attribute_name_list,
        LIST *append_isa_attribute_list,
        char *descending_label );
```

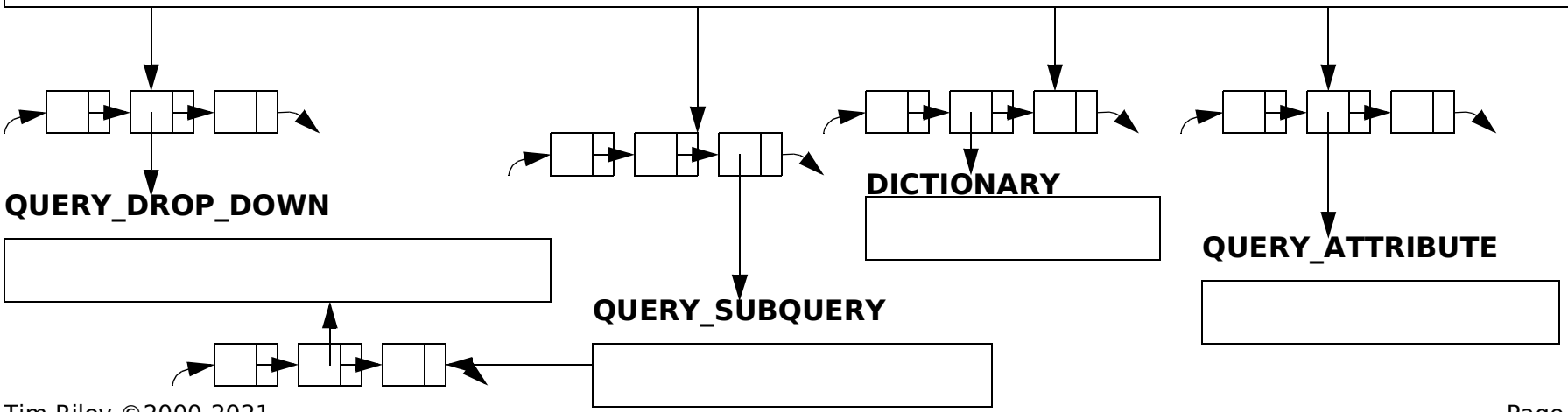
```
enum
relational_operator {
    equals                  not_equal               not_equal_or_null
    less_than               less_than_equal_to      greater_than
    greater_than_equal_to   between                 begins
    contains                not_contains            query_or
    is_null                 not_null
```



APPASERVER Data Structure

QUERY_OUTPUT

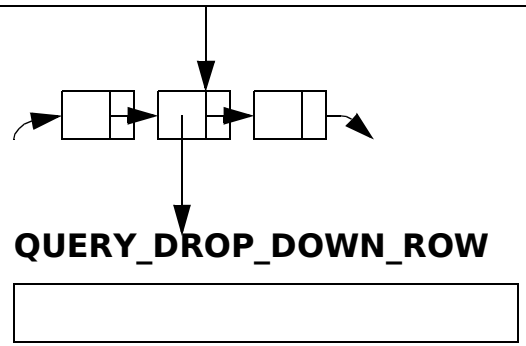
```
/* Usage */
QUERY_OUTPUT *query_simple_output_new(
        DICTIONARY *query_dictionary,
        FOLDER *mto1_folder,
        LIST *ignore_attribute_name_list,
        PROMPT_RECURSIVE *prompt_recursive );

QUERY_OUTPUT *query_edit_table_output_new(
        DICTIONARY *query_dictionary,
        FOLDER *mto1_folder,
        LIST *ignore_attribute_name_list,
        PROMPT_RECURSIVE *prompt_recursive,
        char *attribute_not_null_join,
        char *attribute_not_null_folder_name,
        DICTIONARY *sort_dictionary );

QUERY_OUTPUT *query_detail_output_new(
        LIST *primary_attribute_data_list,
        FOLDER *mto1_folder,
        LIST *ignore_attribute_name_list,
        char *attribute_not_null_join,
        char *attribute_not_null_folder_name,
        DICTIONARY *sort_dictionary );

/* Process */
QUERY_OUTPUT *query_output_calloc(
        void );

LIST *query_drop_down_list(
        exclude_attribute_name_list
                /* includes ignore and exclude lists */,
        mto1_folder,
        query_dictionary );

LIST *query_prompt_recursive_drop_down_list(
        exclude_attribute_name_list
                /* includes ignore and exclude lists */,
        mto1_folder->folder_name,
        prompt_recursive->
                prompt_recursive_folder_list,
        query_dictionary );

LIST *query_attribute_list(
        mto1_folder->append_isa_attribute_list,
        query_dictionary,
        exclude_attribute_name_list
                /* includes ignore and exclude lists */ );

char *query_output_drop_down_where(
        query_drop_down_list() );

char *query_output_drop_down_data_where(
        char *mto1_folder_name,
        char *attribute_name,
        char *data );

char *query_output_attribute_where(
        query_attribute_list() );

char *query_output_join_where(
        folder_name,
        primary_attribute_name_list,
        mto1_folder->
                mto1_isa_related_folder_list );

char *query_output_related_join(
        folder_name,
        primary_attribute_name_list,
        isa_related_folder->folder->folder_name,
        isa_related_folder->
                foreign_attribute_name_list );

char *query_output_where(
        query_output_drop_down_where(),
        query_output_attribute_where(),
        query_output_join_where(),
        attribute_not_null_join );

char *query_output_select(
        mto1_folder->append_isa_attribute_list,
        ignore_attribute_name_list,
        mto1_folder->
                lookup_attribute_exclude_name_list );

char *query_output_from(
        mto1_folder->folder_name,
        mto1_folder->mto1_isa_related_folder_list,
        attribute_not_null_folder_name );

char *query_output_order(
        folder_name,
        mto1_folder->primary_attribute_name_list,
        mto1_folder->append_isa_attribute_list,
        sort_dictionary );

char *query_output_display(
```



QUERY_DROP_DOWN

```
/* Usage */
LIST *query_drop_down_list(
        LIST *exclude_attribute_name_list /* in/out */,
        FOLDER *mto1_folder,
        DICTIONARY *query_dictionary );

QUERY_DROP_DOWN *query_drop_down(
        LIST *exclude_attribute_name_list /* in/iout */,
        char *mto1_folder_name,
        char *one2m_folder_name,
        LIST *one2m_primary_attribute_name_list,
        LIST *mto1_foreign_attribute_name_list,
        LIST *one2m_attribute_list,
        DICTIONARY *query_dictionary );

/* Input */
char *mto1_folder_name;
char *one2m_folder_name;
LIST *one2m_primary_attribute_name_list;
LIST *mto1_foreign_attribute_name_list;
LIST *one2m_attribute_list;
```

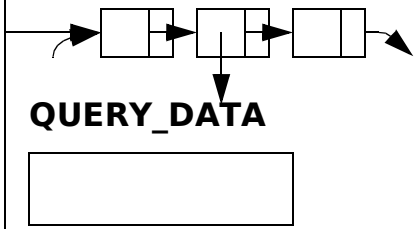
```
/* Process */
QUERY_DROP_DOWN *query_drop_down_new(
        char *mto1_folder_name,
        char *one2m_folder_name,
        LIST *one2m_primary_attribute_name_list,
        LIST *mto1_foreign_attribute_name_list );

LIST *drop_down_row_list =
        query_drop_down_row_list(
                exclude_attribute_name_list,
                mto1_folder_name,
                one2m_folder_name,
                one2m_primary_attribute_name_list,
                mto1_foreign_attribute_name_list,
                one2m_attribute_list,
                query_dictionary,
                dictionary_attribute_name_list_highest_index(
                        query_dictionary,
                        one2m_primary_attribute_name_list ) );
```



APPASERVER Data Structure

QUERY_DROP_DOWN_ROW

```
/* Usage */
LIST *query_drop_down_row_list(
        LIST *exclude_attribute_name_list /* in/iout */,
        char *mto1_folder_name,
        char *one2m_folder_name,
        LIST *one2m_primary_attribute_name_list,
        LIST *mto1_foreign_attribute_name_list,
        LIST *one2m_attribute_list,
        DICTIONARY *query_dictionary,
        int highest_index );

QUERY_DROP_DOWN_ROW *query_drop_down_row(
        LIST *exclude_attribute_name_list /* in/iout */,
        char *mto1_folder_name,
        char *one2m_folder_name,
        LIST *one2m_primary_attribute_name_list,
        LIST *mto1_foreign_attribute_name_list,
        LIST *one2m_attribute_list,
        DICTIONARY *query_dictionary,
        int index );

QUERY_DROP_DOWN_ROW *query_drop_down_row_new(
        char *mto1_folder_name,
        char *one2m_folder_name,
        LIST *one2m_primary_attribute_name_list,
        LIST *mto1_foreign_attribute_name_list,
        LIST *one2m_attribute_list,
        int index );

/* Input */
char *mto1_folder_name;
char *one2m_folder_name;
LIST *one2m_primary_attribute_name_list;
LIST *mto1_foreign_attribute_name_list;
LIST *one2m_attribute_list;
int index;

/* Process */
QUERY_DROP_DOWN_ROW *query_drop_down_row_calloc(
        void );

char *key =
        query_key(
                LIST *mto1_foreign_attribute_name_list,
                int index );

char *operator_name =
        query_dictionary_operator_name(
                query_key(),
                query_dictionary,
                QUERY_RELATION_OPERATOR_STARTING_LABEL );

enum relational_operator relational_operator =
        query_relational_operator(
                query_dictionary_operator_name(),
                (char *)0 /* attribute_datatype */ );

char *data_list_string =
        query_data_list_string(
                query_dictionary,
                key );

LIST *data_string_list =
        query_data_string_list(
                data_list_string );

LIST *query_data_list =
        query_drop_down_query_data_list(
                one2m_primary_attribute_name_list,
                mto1_foreign_attribute_name_list,
                one2m_attribute_list,
                data_string_list );
```



QUERY_ATTRIBUTE

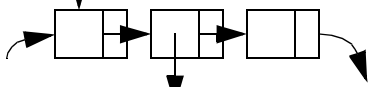
```
/* Usage */
LIST *query_attribute_list(
        LIST *append_isa_attribute_list,
        DICTIONARY *query_dictionary,
        LIST *exclude_attribute_name_list );

QUERY_ATTRIBUTE *query_attribute_new(
        char *attribute_name,
        char *folder_name,
        char *datatype,
        enum relation_operator relation_operator;
        QUERY_DATA *from_data;
        QUERY_DATA *to_data;
        int primary_key_index );

/* Input */
char *attribute_name;
char *folder_name;
char *datatype;
enum relational_operator relational_operator;
QUERY_DATA *from_data;
QUERY_DATA *to_data;
int primary_key_index;

/* Process */
char *query_attribute_list_display(
        char *subquery_folder_name,
        LIST *query_attribute_list );

boolean query_attribute_exists(
        LIST *query_attribute_list,
        char *attribute_name );

boolean query_attribute_date_time_between(
        QUERY_ATTRIBUTE **date_between_attribute,
        QUERY_ATTRIBUTE **time_between_attribute,
        LIST *query_attribute_list() );

char *query_attribute_between_date_time_where(
        char *date_attribute_name,
        char *time_attribute_date,
        QUERY_DATA *date_from_data,
        QUERY_DATA *time_from_data,
        QUERY_DATA *date_to_data,
        QUERY_DATA *time_to_data,
        char *folder_name );

boolean query_attribute_is_date(
        char *datatype );

boolean query_attribute_is_time(
        char *datatype );

boolean query_attribute_is_number(
        char *datatype );

boolean query_attribute_is_date_time(
        char *datatype );
```

QUERY_DATA

```
/* Usage */
LIST *query_drop_down_query_data_list(
        LIST *one2m_primary_attribute_name_list,
        LIST *mto1_foreign_attribute_name_list,
        LIST *one2m_attribute_list,
        LIST *data_string_list );

QUERY_DATA *query_data_new(
        char *attribute_name,
        char *attribute_datatype,
        char *security_sql_injection_escape(
                security_replace_special_characters(
                        string_trim_number_characters(
                                char *data,
                                char *attribute_datatype ) ) ) );

/* Input */
char *attribute_name;
char *attribute_datatype;
char *escaped_replaced_data;

/* Process */
char *escaped_replaced_data =
        query_data_convert_date_international(
                char *attribute_datatype,
                char *escaped_replaced_data );
```

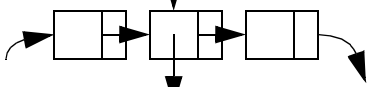
UPDATE_DATABASE

```
/* Input */
char *application_name;
char *session;
char *login_name;
char *role_name;
DICTIONARY *post_dictionary;
DICTIONARY *file_dictionary;

/* Process */
FOLDER *folder =
        folder_fetch(
                folder_name,
                1 /* fetch_attribute_list */,
                1 /* fetch_one2m_recursive_relation_list */,
                1 /* fetch_mto1_isa_recursive_relation_list */,
                0 /* not fetch_mto1_relation_list */,
                1 /* fetch_process */ );

LIST *update_database_update_row_list(
        /* Sets preupdate_$attribute_name for trigger */
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        char *folder_name,
        folder->attribute_list,
        folder->one2m_recursive_relation_list,
        folder->post_change_process );
```



UPDATE_ROW

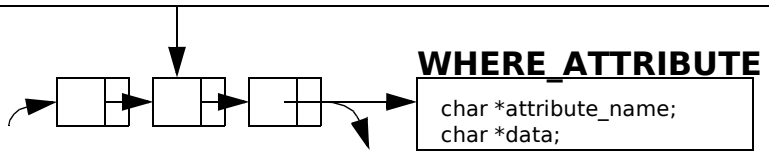
```
/* Usage */
UPDATE_ROW *update_database_update_row(
        /* Sets preupdate_$attribute_name for trigger */
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        char *folder_name,
        LIST *attribute_list,
        LIST *one2m_recursive_relation_list,
        PROCESS *post_change_process,
        int row );

/* Process */
boolean changed_primary_key;

LIST *update_folder_list =
        update_database_update_folder_list(
                primary_update_folder->primary_data_list,
                primary_update_folder->changed_attribute_list,
                primary_update_folder->changed_primary_key,
                one2m_recursive_relation_list,
                mto1_isa_recursive_relation_list,
                primary_update_folder );
```

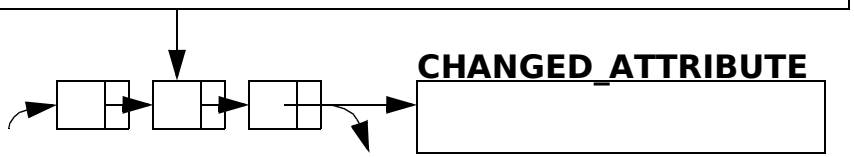


UPDATE_FOLDER

UPDATE_FOLDER

```
/* Usage */
LIST *update_database_update_folder_list(
        int row,
        LIST *primary_data_list,
        LIST *primary_changed_attribute_list,
        boolean changed_primary_key,
        LIST *one2m_recursive_relation_list,
        UPDATE_FOLDER *primary_update_folder );

/* Process */
boolean changed_primary_key;

UPDATE_FOLDER *update_primary_update_folder(
        /* Sets preupdate_$attribute_name for trigger */
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        char *folder_name,
        LIST *attribute_list,
        PROCESS *post_change_process,
        int row );

UPDATE_FOLDER *update_secondary_update_folder(
        mto1_folder->folder_name,
        relation_foreign_attribute_name_list(),
        primary_data_list,
        primary_changed_attribute_list );

/* Process */
LIST *primary_attribute_name_list =
        attribute_primary_name_list();

LIST *relation_foreign_attribute_name_list =
        relation_foreign_attribute_name_list();

LIST *update_folder_changed_attribute_list(
        boolean *changed_primary_key,
        /* Sets preupdate_$attribute_name for trigger */
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        LIST *attribute_list,
        int row );

LIST *update_folder_primary_data_list(
        DICTIONARY *file_dictionary,
        LIST *primary_attribute_name_list,
        int row );

LIST *update_folder_where_attribute_name_list(
        LIST *primary_attribute_name_list,
        LIST *relation_foreign_attribute_name_list );

LIST *update_folder_where_attribute_list(
        LIST *update_folder_where_attribute_name_list(),
        LIST *update_folder_primary_data_list() );

char *update_folder_where_clause(
        update_folder_where_attribute_list() );

char *update_folder_set_clause(
        update_folder_changed_attribute_list() );

int update_folder_count(
        update_folder_where_clause() );
```

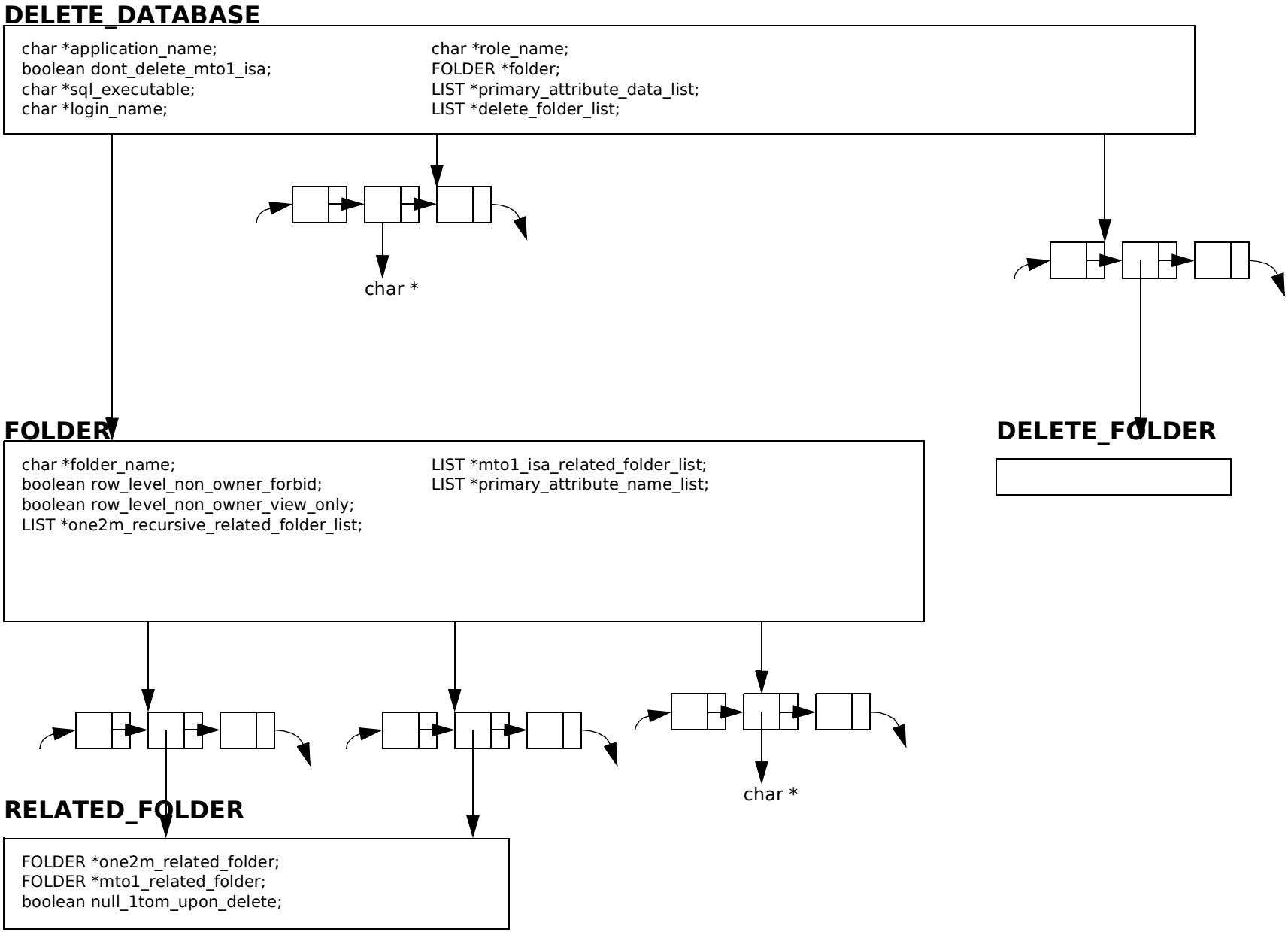


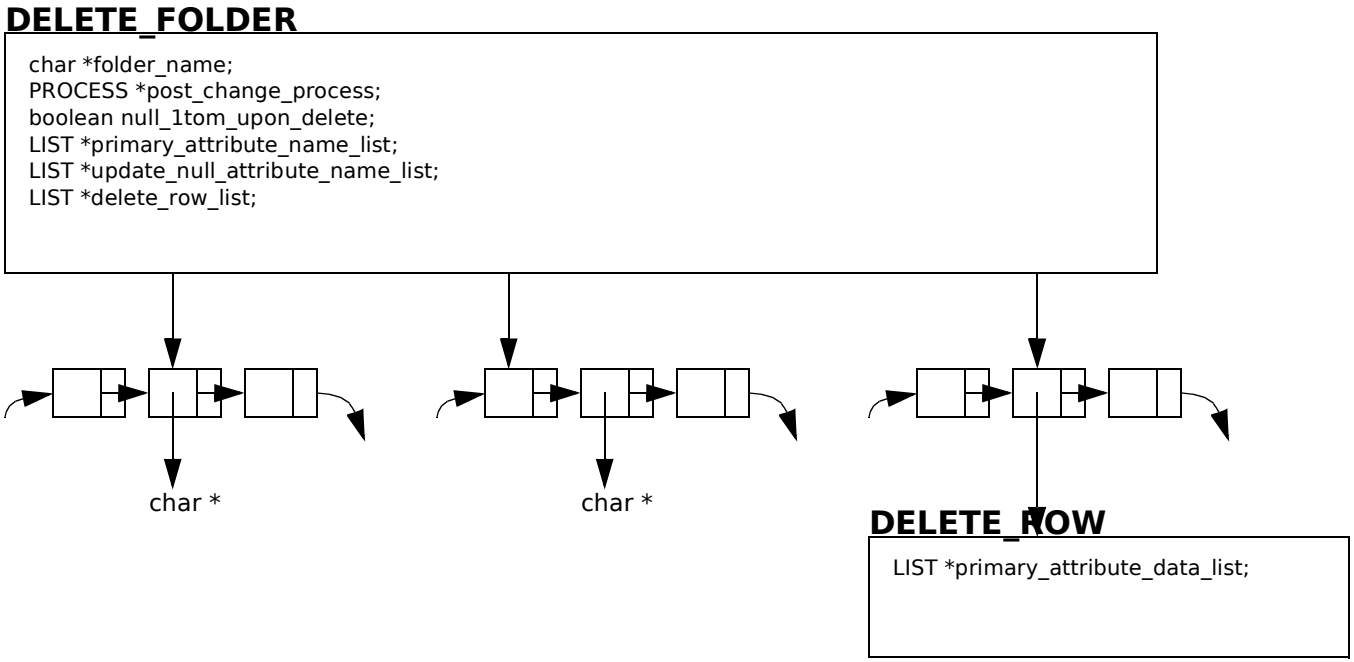


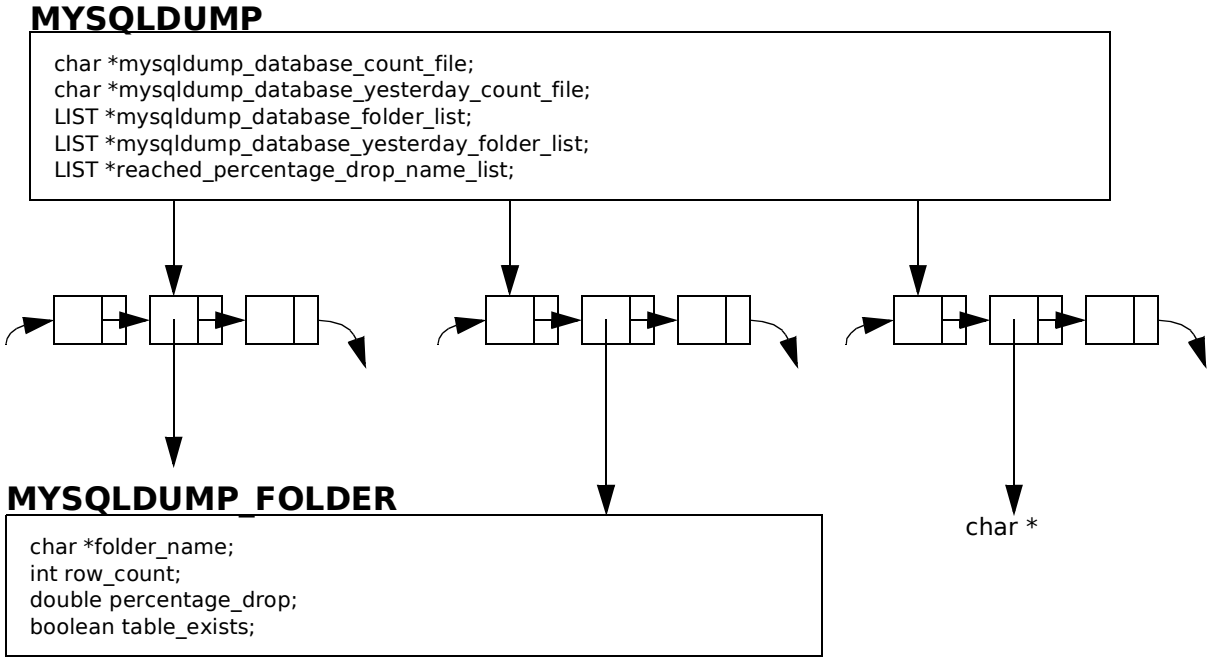
CHANGED_ATTRIBUTE

```
/* Usage */
LIST *update_changed_attribute_list(
        boolean *changed_primary_key,
        /* Sets preupdate_$attribute_name for trigger */
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        LIST *attribute_list,
        int row );

char *attribute_name;
char *attribute_datatype;
char *old_data;
char *escaped_replaced_new_data;

/* Process */
int changed_primary_key_index;
```

```
CHANGED_ATTRIBUTE *update_changed_attribute(
        /* Sets preupdate_$attribute_name for trigger */
        DICTIONARY *post_dictionary,
        DICTIONARY *file_dictionary,
        ATTRIBUTE *attribute,
        int row );

CHANGED_ATTRIBUTE *update_changed_attribute_new(
        char *attribute_name,
        char *attribute_datatype,
        char *old_data,
        char *security_sql_injection_escape(
                security_replace_special_characters(
                        string_trim_number_characters(
                                char *new_data,
                                char *attribute_datatype ) ) ) );
```


DELETE_DATABASE
char *application_name;
boolean dont_delete_mto1_isa;
char *sql_executable;
char *login_name;
char *role_name;
FOLDER *folder;
LIST *primary_attribute_data_list;
LIST *delete_folder_list;
char *
FOLDER
char *folder_name;
boolean row_level_non_owner_forbid;
boolean row_level_non_owner_view_only;
LIST *one2m_recursive_related_folder_list;
LIST *mto1_isa_related_folder_list;
LIST *primary_attribute_name_list;
DELETE_FOLDER
char *
RELATED_FOLDER
FOLDER *one2m_related_folder;
FOLDER *mto1_related_folder;
boolean null_1tom_upon_delete;


DELETE_FOLDER
char *folder_name;
PROCESS *post_change_process;
boolean null_1tom_upon_delete;
LIST *primary_attribute_name_list;
LIST *update_null_attribute_name_list;
LIST *delete_row_list;
char *
char *
DELETE_ROW
LIST *primary_attribute_data_list;


MYSQLDUMP
char *mysqldump_database_count_file;
char *mysqldump_database_yesterday_count_file;
LIST *mysqldump_database_folder_list;
LIST *mysqldump_database_yesterday_folder_list;
LIST *reached_percentage_drop_name_list;
MYSQLDUMP_FOLDER
char *folder_name;
int row_count;
double percentage_drop;
boolean table_exists;
char *

SECURITY

```
enum password_function password_function;{ no_encryption
                                          old_password_function
                                          regular_password_function
                                          sha2_function

enum password_function
        security_database_password_function(
                char *database_password );

char *security_encrypted_password(
        char *application_name,
        char *password_sql_injection_escape,
        enum password_function password_function );

char *security_mysql_version( void );

enum password_function
        security_mysql_version_password_function(
                security_mysql_version() );

char *security_encryption_select_clause(
        enum password_function password_function,
        char *password );

boolean security_password_match(
        database_password,
        security_encrypted_password(
                application_name,
                security_sql_injection_escape(
                        typed_in_password ),
                security_database_password_function() );

boolean security_password_encrypted(
        char *database_password );

char *security_replace_special_characters(
        char *data );

char *security_sql_injection_escape(
        char *data )
{
        char destination[ QUERY_WHERE_BUFFER ];

        return strdup(
                string_escape_character_array(
                        destination,
                        data,
                        “‘’$;%$=” ) );
}
```

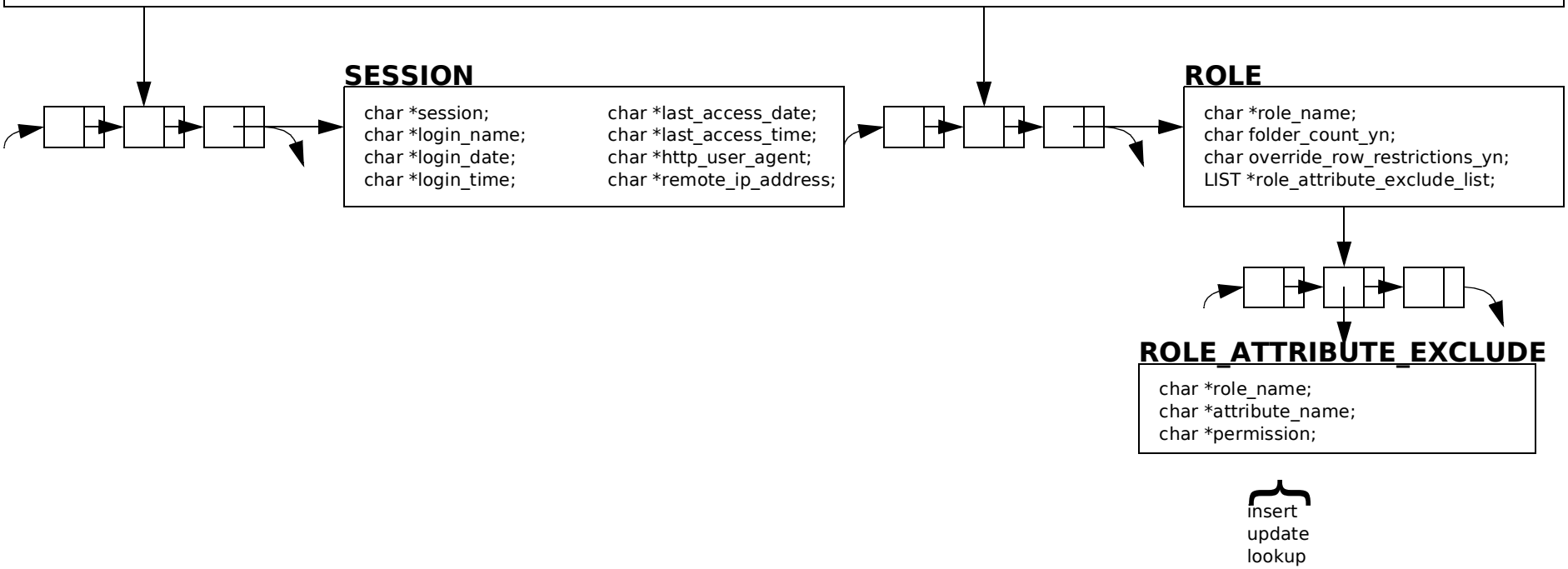
STRING

```
boolean string_strcpy(
        char *destination,
        char *source,
        int buffer_size );

boolean string_strcmp(
        char *string1,
        char *string2 );

char *string_escape_character(
        char *destination,
        char *data,
        char character );

char *string_escape_character_array(
        char *destination,
        char *data,
        char *character_array );

/* Position is 1 based */

int string_character_position(
        char *data,
        char character );

char *string_trim_character(
        char *data,
        char character );

char *string_trim_character_array(
        char *data,
        char *character_array );

char *string_trim_number_characters(
        char *number,
        char *attribute_datatype )
{
        return string_trim_character_array(
                number,
                “,$” );
}
```

APPASERVER Data Structure

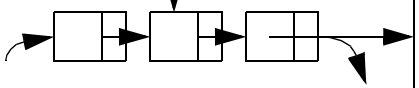
APPASERVER_USER

```
/* Usage */
APPASERVER_USER *appaserver_user_fetch(
        char *login_name,
        boolean fetch_role_list,
        boolean fetch_attribute_exclude_list,
        boolean fetch_session_list );

/* Input */
char *login_name;
char *full_name;
char *typed_in_password;
char *database_password;
char *user_date_format;

/* Process */
APPASERVER_USER *appaserver_user_calloc( void );

APPASERVER_USER *appaserver_user_parse(
        char *input,
        boolean fetch_role_list,
        boolean fetch_attribute_exclude_list,
        boolean fetch_session_list );

LIST *appaserver_user_role_list(
        char *login_name );

LIST *appaserver_user_session_list(
        char *login_name );

FILE *appaserver_user_update_open( void );

void appaserver_user_update(
        appaserver_user_update_open(),
        security_encrypted_password(
                application_name,
                security_sql_injection_escape(
                        database_password ),
                security_mysql_version_password_function(
                        security_mysql_version() ) ),
        login_name );
```



PAIR_ONE2M

```
char *many_folder_name;
char *one_folder_name;

PAIR_ONE2M *pair_one2m_prompt_form_new(
        char *one_folder_name,
        char *keystrokes_save_function );

LIST *pair_one2m_prompt_form_folder_list(
        char *keystrokes_save_function,
        LIST *relation_one2m_pair_relation_list() );

char *pair_one2m_prompt_element_name(
        char *PAIR_ONE2M_PREFIX,
        char *folder_label );

boolean pair_one2m_participating(
        DICTIONARY *pair_one2m_dictionaryy );

PAIR_ONE2M *pair_one2m_post_new(
        DICTIONARY *pair_one2m_dictionary );

boolean pair_one2m_duplicate(
        char *MYSQL_DUPLICATE_ERROR_MESSAGE_KEY,
        char *message );

void pair_one2m_duplicate_set(
        DICTIONARY *pair_one2m_dictionary,
        char *PAIR_ONE2M_DUPLICATE_KEY );

boolean pair_one2m_inserted_duplicate(
        char *PAIR_ONE2M_DUPLICATE_KEY,
        DICTIONARY *pair_one2m_dictionary );

LIST *pair_one2m_fulfilled_folder_name_list(
        char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
        DICTIONARY *pair_one2m_dictionary );

char *pair_one2m_folder_name(
        char *folder_label,
        DICTIONARY *pair_one2m_dictionary );

void pair_one2m_folder_set(
        DICTIONARY *pair_one2m_dictionary,
        char *folder_label,
        char *folder_name );

char *pair_one2m_next_folder_name(
        LIST *pair_one2m_fulfilled_folder_name_list(),
        LIST *relation_one2m_pair_relation_list(),
        char *one_folder_name,
        char *many_folder_name );

DICTIONARY  *pair_one2m_fulfilled_dictionary(
        DICTIONARY *pair_one2m_dictionary,
        char *PAIR_ONE2M_FULFILLED_LIST_LABEL,
        LIST *pair_one2m_fulfilled_folder_name_list );

/* Insert form */
PAIR_ONE2M *pair_one2m_insert_form_new(
        DICTIONARY *pair_one2m_dictionary );
```



PAIR_ONE2M_FOLDER

```
/* Usage */
pair_one2m_folder_new(
        char *many_folder_name );

char *many_folder_name;

char *pair_folder_element_copy_function(
        prompt_many_element_name,
        many_folder_name,
        pair_one2m_prompt_form_element_name(),
        folder_name );

char *pair_folder_button_string(
        many_folder_name,
        keystrokes_save_function,
        pair_folder_element_copy_function() );
```

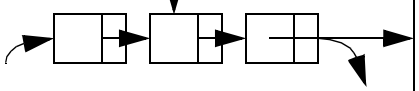
VERTICAL_NEW_BUTTON

```
char *many_folder_name;
char *one_folder_name;

VERTICAL_NEW_BUTTON *vertical_new_button_calloc(
        void );

char *vertical_new_button_one_element_name(
        char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
        char *one_folder_name );

char *vertical_new_button_many_element_name(
        char *VERTICAL_NEW_BUTTON_MANY_LABEL );

ELEMENT_APPASERVER *vertical_new_button_element(
        char *one_folder_name,
        LIST *role_folder_insert_list,
        char *form_name,
        char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
        char *VERTICAL_NEW_BUTTON_DISPLAY,
        char *onclick_keystrokes_save_string );

char *vertical_new_button_dictionary_one_folder_name(
        char *VERTICAL_NEW_BUTTON_ONE_PREFIX,
        DICTIONARY *non_prefixed_dictionary );

char *vertical_new_button_dictionary_folder_name(
        char *hidden_label,
        DICTIONARY *non_prefixed_dictionary );

void vertical_new_button_dictionary_set(
        DICTIONARY *non_prefixed_dictionary,
        char *hidden_label,
        char *folder_name );
```

INSERT_DATABASE

```
char *application_name;
char *session;
char *folder_name;
LIST *primary_attribute_name_list;
LIST *attribute_name_list;
LIST *data_list;
DICTIONARY *row_dictionary;
LIST *ignore_attribute_name_list;
boolean insert_row_zero_only;
boolean dont_remove_tmp_file;
int rows_inserted;
char *post_change_process;
char *login_name;
char *tmp_file_name;
FILE *tmp_file;
LIST *attribute_list;
```

```
LIST *attribute_data_list =
        insert_database_attribute_data_list(
                row_dictionary,
                row,
                ignore_attribute_name_list,
                primary_attribute_name_list,
                attribute_name_list,
                mto1_related_folder_list,
                common_non_primary_attribute_name_list,
                application_name,
                attribute_list );

LIST *insert_attribute_name_list =
        insert_database_attribute_name_list(
                insert_database_attribute_data_list() );
```



INSERT_DATABASE_ATTRIBUTE_DATA

```
/* Usage */
INSERT_DATABASE_ATTRIBUTE_DATA *
        insert_database_attribute_data_new(
                char *attribute_name,
                char *security_sql_injection_escape(
                        security_replace_special_characters(
                                string_trim_number_characters(
                                        char *data,
                                        char *attribute_datatype ) ) ) );

char *attribute_name;
char *escaped_replaced_data;
```

PROCESS_GENERIC (1)

```
/* Usage */
PROCESS_GENERIC *process_generic_fetch(
        process_set_name,
        process_name,
        login_name,
        output_medium_string,
        post_dictionary );

char *process_set_name;
char *login_name;
char *output_medium_string;
DICTIONARY *post_dictionary;

/* Process */
char *process_name =
        process_generic_process_name(
                this->process_set_name,
                post_dictionary );

char *value_folder_name =
        process_generic_value_folder_name(
                this->process_name,
                this->process_set_name );

PROCESS_GENERIC_VALUE_FOLDER *value_folder =
        process_generic_value_folder_fetch(
                this->value_folder_name );

PROCESS_GENERIC_PARAMETER *parameter =
        process_generic_parameter_parse(
                this->output_medium_string,
                this->post_dictionary,
                this->value_folder->datatype->aggregation_sum,
                this->value_folder->primary_attribute_name_list,
                this->value_folder->date_attribute_name,
                this->value_folder->time_attribute_name );

char *process_generic_select(
        this->value_folder->primary_attribute_name_list,
        this->parameter->value_attribute_name );

char *process_generic_date_where(
        this->value_folder->date_attribute_name,
        this->parameter->begin_date,
        this->parameter->end_date );

char *process_generic_where(
        this->value_folder->foreign_folder->foreign_attribute_name_list,
        this->value_folder->foreign_folder->foreign_attribute_data_list,
        process_generic_date_where() );

char *process_generic_system_string(
        process_generic_select(),
        this->value_folder_name,
        process_generic_where(),
        this->parameter->aggregate_level,
        this->parameter->aggregate_statistic,
        this->parameter->accumulate,
        this->parameter->date_piece,
        this->parameter->time_piece,
        this->parameter->value_piece,
        this->parameter->end_date,
        this->value_folder->datatype->unit /* datatype_unit */,
        this->value_folder->foreign_folder->unit /* foreign_folder_unit */,
        this->parameter->units_converted );

char *process_generic_heading(
        this->value_folder->primary_attribute_name_list,
        this->value_folder->value_attribute_name,
        this->value_folder->datatype->unit /* datatype_unit */,
        this->value_folder->foreign_folder->unit /* foreign_folder_unit */,
        this->parameter->units_converted,
        this->parameter->aggregate_level,
        this->parameter->aggregate_statistic,
        this->parameter->accumulate );

char *process_generic_subtitle(
        this->value_folder_name,
        this->parameter->begin_date,
        this->parameter->end_date,
        this->parameter->aggregate_level,
        this->parameter->aggregate_statistic
        additional_message );
```

PROCESS_GENERIC_PARAMETER

PROCESS_GENERIC_VALUE_FOLDER

PROCESS_GENERIC (2)

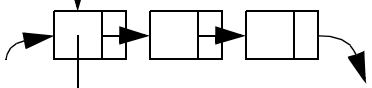
```
char *units_label(
        units,
        units_converted,
        enum aggregate_statistic );

enum date_convert_format process_generic_date_convert_format(
        login_name );

boolean process_generic_no_cycle_color(
        role_name );
```

PROCESS_GENERIC_PARAMETER

```
/* Attributes */
char *output_medium_string;
enum process_generic_medium output_medium;{ text_file,
                                            output_medium_stdout,
                                            table,
                                            spreadsheet,
                                            gracechart,
                                            output_medium_unknown
char *begin_date;
char *end_date;
enum aggregate_level aggregate_level;
enum aggregate_statistic aggregate_statistic;
boolean accumulate;
char *email_address;
char *units_converted;
pidt process_id;
int date_piece;
int time_piece;
int value_piece;

/* Usage */
PROCESS_GENERIC_PARAMETER *process_generic_parameter_parse(
        process_generic->output_medium_string,
        process_generic_>post_dictionary,
        process_generic->value_folder->datatype->aggregation_sum,
        value_folder_primary_attribute_name_list,
        value_folder_date_attribute_name,
        value_folder_time_attribute_name );
```

PROCESS_GENERIC_VALUE_FOLDER

```
/* Usage */
PROCESS_GENERIC_VALUE_FOLDER *value_folder =
        process_generic_value_folder_fetch(
                process_generic->value_folder_name,
                process_generic->post_dictionary );

/* Input */
char *value_folder_name;
DICTIONARY *post_dictionary;

/* Process */
PROCESS_GENERIC_VALUE_FOLDER *value_folder_parse(
        input );

char *datatype_folder_name;
char *foreign_folder_name;
char *datatype_attribute_name;
char *date_attribute_name;
char *time_attribute_name;
char *value_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_unit;
boolean foreign_exists_unit;

LIST *primary_attribute_name_list =
        folder_primary_attribute_name_list_fetch(
                this->value_folder_name );

PROCESS_GENERIC_DATATYPE *datatype =
        process_generic_datatype_fetch(
                this->datatype_folder_name,
                this->datatype_attribute_name,
                this->exists_aggregation_sum,
                this->exists_bar_graph,
                this->exists_scale_graph_zero,
                this->datatype_exists_units,
                this->post_dictionary );

PROCESS_GENERIC_FOREIGN_FOLDER *foreign_folder =
        process_generic_foreign_folder_fetch(
                this->foreign_folder_name,
                this->foreign_exists_unit,
                this->post_dictionary );

LIST *process_generic_datatype_compare_list(
        process_generic->post_dictionary,
        this->exists_aggregation_sum,
        this->exists_bar_graph,
        this->exists_scale_graph_zero,
        this->datatype_exists_unit );
```



PROCESS_GENERIC_DATATYPE

PROCESS_GENERIC_FOREIGN_FOLDER

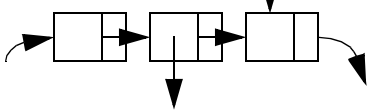
char *

PROCESS_GENERIC_FOREIGN_FOLDER

```
char *foreign_folder_name;
FOLDER *foreign_folder;

/* Usage */
PROCESS_GENERIC_FOREIGN_FOLDER *process_generic_foreign_folder_fetch(
        process_generic->value_folder->foreign_folder_name,
        process_generic->post_dictionary,
        process_generic->value_folder->foreign_exists_unit );

LIST *foreign_attribute_name_list =
        foreign_folder->primary_attribute_name_list;

LIST *foreign_attribute_data_list =
        dictionary_key_list_fetch(
                this->foreign_attribute_name_list
                        /* key_list */,
                this->post_dictionary );

char *unit = (foreign_exists_unit) ? list_last( foreign_attribute_data_list ) : (char *)0;
```

FOLDER

```
char *folder_name;
LIST *primary_attribute_name_list
```

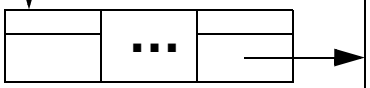


PROCESS_GENERIC_DATATYPE

```
/* Usage */
PROCESS_GENERIC_DATATYPE *process_generic_datatype_fetch(
        process_generic_value_folder->datatype_folder_name,
        process_generic_value_folder->datatype_attribute_name,
        process_generic_value_folder->exists_aggregation_sum,
        process_generic_value_folder->exists_bar_graph,
        process_generic_value_folder->exists_scale_graph_zero,
        process_generic_value_folder->datatype_exists_units,
        process_generic_value_folder->post_dictionary );

/* Input */
char *datatype_folder_name;
char *datatype_attribute_name;
boolean exists_aggregation_sum;
boolean exists_bar_graph;
boolean exists_scale_graph_zero;
boolean datatype_exists_units;
DICTIONARY *post_dictionary;

/* Process */
char *datatype_name;
boolean aggregation_sum;
boolean bar_graph;
boolean scale_graph_zero;
char *unit;
```

```
char *process_generic_datatype_select(
        this->datatype_attribute_name,
        this->exists_aggregation_sum,
        this->exists_bar_graph,
        this->exists_scale_graph_zero,
        this->datatype_exists_unit );

char *datatype_name =
        dictionary_fetch(
                this->datatype_attribute_name,
                this->post_dictionary );

char *process_generic_datatype_where(
        this->datatype_attribute_name,
        this->datatype_name );

char *process_generic_datatype_system_string(
        process_generic_datatype_select(),
        process_generic_datatype->datatype_folder_name,
        process_generic_datatype_where() );

PROCESS_GENERIC_DATATYPE *process_generic_datatype_parse(
        string_pipe_fetch(
                process_generic_datatype_system_string() )
                        /* input_buffer */ );
```

key = date or
key = date^time



PROCESS_GENERIC_VALUE

```
char *date;
char *time;
char *unit;
double value;
boolean is_null;
double accumulate
```

```
PROCESS_GENERIC_VALUE *process_generic_value_parse(
        double *accumulate_value,
        char *input,
        char *time_attribute,
        boolean unit_value_folder,
        boolean accumulate_boolean,
        enum aggregate_level );
```